由於需要自行針對 mockdata 給假資料，若是更改或新增 stack 都要進行改動，所以希望可以取代掉中間這段(呼叫 aws api 取得資料讓流程進行下去最後取得資源)

改動

發現有專門用於 CI 的 image 他們負責做的事情就是讓 pulumi aws provider 可以把 AWS API endpoint 指向一個假的 stack 讓他代替 AWS 回應某些 API 目前用的是 ministack 這個專案

目前整個 workflow 的運作是

- 開 pr 到 main 會觸發檢查
- 會掃描 `Pulumi.<stack>.yaml`，並把 stack 讀出來確認有哪些 stack
- 每個 stack 開一個獨立的 MiniStack，本來想過共用同一個，但發現 GitHub Actions 的每個 `services:` 會分開，不能共用資源，如果強行共用資源，會變成依序進行檢查，可能會有點慢，而且有錯誤的話不太好看
- 啟動 ministack 會提供假的 AWS api
- runner 上的 AWS CLI 跟 pulumi provider 會透過 `AWS_ENDPOINT_URL=http://127.0.0.1:4566` (設定在 workflow 裡面) 連到 ministack，並且會用一組 test/test 的 token 來連

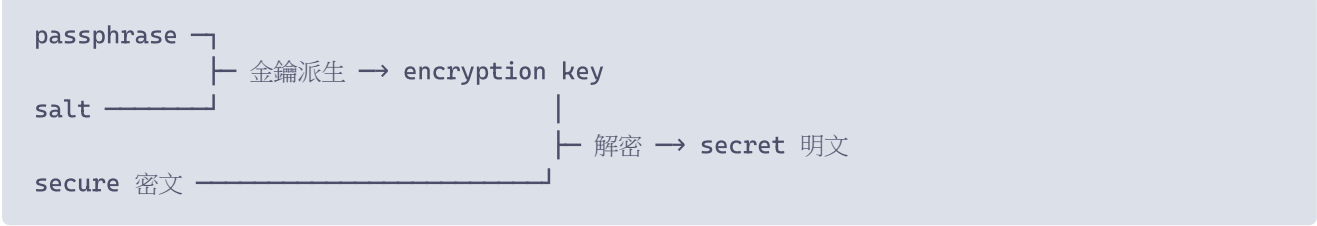
```
AWS_ACCESS_KEY_ID: test
AWS_SECRET_ACCESS_KEY: test
```

- 接下來就看系統需要哪些 aws api 會自行去 call，但在這邊遇到一個很尷尬的問題，其實我原本想解決的問題也還是存在，只是比較少一點而已，比如說會需要查詢 route53 的 zone id，舉例來說需要查 `threatreveal.org` 這個 domain 的 zone_id ，所以我的 ministack 裡面還是需要模擬 threatreveal.org 對應的 zone_id、ARN、Private Zone、名稱跟一些資料...所以還是要根據啟動的服務來新增一些資料...
- 接下來建立一次性的 Pulumi stack，他看起來會像第一次建一樣(空的 stack)，所以等等的 preview 都會看起來像 create 新的 stack
- encryptionsalt 刪掉(可去 `## 針對 secret` 看原因)
- 接下來會執行 pulumi preview 整理成 OPA 能處理的資料然後檢查

針對 secret

如果在 yaml 中遇到 secret，我們無法依靠假的 aws api 來解密，簡單講就是

- passphrase 會存在本機的環境變數裡面或是 GitHub Secret
- encryptionsalt 會在 Pulumi.production.yaml
- encryption key 在 Pulumi 執行期間的記憶體
- secure: 在Pulumi.production.yaml



但後來想了一下我們好像也可以不解密，直接把 salt 刪掉，那 pulumi 看不到既有的 encryptionsalt 會覺得這是一個全新的 stack 的加密環境(正常來講會是，Pulumi 看到有 encryptionsalt 就知道這是已經建過 passphrase 的加密環境，所以要驗證提供的 passphrase 是否跟他匹配)，反過來說所以他會用我寫在 workflow 裡面的 passphrase 初始化，他會建立新的 salt、encryption Key，那如果接下來的工作不需要用這個 secure 來做的話就沒事，但如果需要還是會失敗...

cis

仔細看之後發現，rego_policy_libraries/benchmarks/cis/aws/cis_aws_foundations.rego at main · ynotbhatc/rego_policy_libraries 這個 opa 裡面有很多不是 IaC 的東西，上網查了一下其他人的標準做法，發現大部分如果是拿 CIS 檢查 IaC 的人都會把 CIS 當中不屬於 IaC 的項目刪掉不假裝有覆蓋，所以我在想我也可以比照辦理，透過工具先整理出以下項目，整理完之後不知道為甚麼有些項目沒有被納入判定，好像是因為有些規則要跟其他規則一起用?但翻過 github 也沒有看到它的規則跟其他規則一起用啊???

但我現在如果直接把下面這份規則套入那應該會有很多地方不合規，可能要先檢查是否有哪些服務，然後再套用相對應的規則嗎?

控制項目	Rego 檢查條件	政策狀態
2.1.1 CloudTrail S3 存取日誌	`bucket.is_cloudtrail_bucket == true` 且未啟用 `bucket.access_logging_enabled`	納入判定
3.1 CloudTrail 多區域啟用	未啟用 `input.cloudtrail.multi_region_enabled`	納入判定
4.1 未授權 API 呼叫監控	Filter Pattern 包含 `ERROR`、`Denied`，且 Alarm 與 Filter 的 `metric_name` 相同	納入判定
5.1 NACL 管理連接埠限制	NACL 允許 `0.0.0.0/0` 存取 `22` 或 `3389`	納入判定
AWS Support IAM Role	Role 名稱包含 `support`，並掛載 `AWSSupportAccess`	只定義函式
Console 登入未使用 MFA	Filter Pattern 包含 `ConsoleLogin`、`MfaUsed`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
Root 帳號使用	Filter Pattern 包含 `root`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
IAM Policy 變更	Filter Pattern 包含 `DeleteGroupPolicy`、`DeleteRolePolicy`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
CloudTrail 設定變更	Filter Pattern 包含 `CreateTrail`、`UpdateTrail`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
Console 驗證失敗	Filter Pattern 包含 `ConsoleLogin`、`Failed`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
KMS 金鑰停用或刪除	Filter Pattern 包含 `DisableKey`、`ScheduleKeyDeletion`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
S3 Bucket Policy 變更	Filter Pattern 包含 `PutBucketAcl`、`PutBucketPolicy`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
AWS Config 設定變更	Filter Pattern 包含 `PutConfigurationRecorder`、`StopConfigurationRecorder`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式

控制項目	Rego 檢查條件	政策狀態
Security Group 變更	Filter Pattern 包含 `AuthorizeSecurityGroupIngress`、`AuthorizeSecurityGroupEgress`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
Network ACL 變更	Filter Pattern 包含 `CreateNetworkAcl`、`CreateNetworkAclEntry`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
Network Gateway 變更	Filter Pattern 包含 `CreateCustomerGateway`、`DeleteCustomerGateway`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
Route Table 變更	Filter Pattern 包含 `CreateRoute`、`CreateRouteTable`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
VPC 變更	Filter Pattern 包含 `CreateVpc`、`DeleteVpc`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式
AWS Organizations 變更	Filter Pattern 包含 `AcceptHandshake`、`AttachPolicy`，且 Alarm 與 Filter 的 `metric_name` 相同	只定義函式

小結

假資料原本是寫在我自己的腳本裡，改成在 CI 用 ministack 起一個假的 AWS endpoint，AWS CLI 跟 pulumi provider 靠 `AWS_ENDPOINT_URL` 指過去，這樣就能跑真正的 pulumi preview，不用再自己去攔 SDK。

但原本想解決的問題還是存在，只是比較少一點。比如要查 route53 的 zone id，ministack 裡面還是要先模擬 threatreveal.org 對應的 zone_id、ARN、Private Zone 這些資料，還是得看啟動哪些服務去補。差別是這些資料現在是餵給假的 aws api，不是我自己寫的那張查表。

另外兩個地方是這樣決定的

- 每個 stack 開一個獨立的 ministack。本來想共用同一個，但 GitHub Actions 的 `services:` 會分開，硬要共用就變成依序檢查，有點慢而且出錯不好看
- secret 不解密，直接把 Pulumi.production.yaml 的 encryptionsalt 刪掉，讓 pulumi 當成全新的加密環境，用 workflow 裡的 passphrase 重新初始化。前提是後面不需要用到那個 secure，需要的話還是會失敗

CIS 這邊方向確定了，那份 rego 裡面有很多不是 IaC 的東西，preview 根本看不到，所以照其他人的做法把不屬於 IaC 的項目刪掉。剩下的問題是直接套下去會有很多地方不合規，可能要先看 stack 裡有哪些服務，再決定套哪些規則。

參考資料

- ministackorg/ministack(本地 AWS emulator，單一 port 4566)
- Infrastructure as Code | MiniStack Docs(Pulumi / Terraform 的 endpoint 設定)
- Service-specific endpoints - AWS SDKs and Tools(`AWS_ENDPOINT_URL`)
- Pulumi AWS Provider(`endpoints` 設定)
- Secrets Handling | Pulumi Docs(passphrase、encryptionsalt、secure)
- About service containers - GitHub Docs(`services:`)

- CIS Amazon Web Services Foundations Benchmark